

Royaume du Maroc

Université ...

École Nationale des Sciences Appliquées – Ville ...

Projet d'Intégration

Filière : Génie Informatique

1ère année

Année : 2025/2026

Propel

**Générateur de CV Intelligent avec Pipeline d'Agents
IA**

Rapport Fonctionnel

Présenté par :

Nom et Prénom	Rôle
Bilal Magguilej	Chef d'équipe, Développeur Fullstack, Responsable CI/CD, Dockerisation, Validation des PR
Mouhssine El Haouary	Développeur Fullstack, Responsable Infrastructure Cloud
Somia Jdia	Développeur Fullstack, Responsable Infrastructure Locale
Nourelhouda Ouhba	Développeur Fullstack, Responsable Sécurité
Mohssine El Addaoui	Développeur Fullstack, Responsable Qualité (Cypress, k6)
Salman El-Ghrich	Développeur Fullstack, Responsable Monitoring (Prometheus, Grafana)

Encadré par : M. GHAILANI

Date de soutenance : 06 Juin 2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réussite de ce projet d'intégration.

Nous remercions chaleureusement **M. GHAILANI**, professeur responsable du module, pour son encadrement, ses orientations précieuses et son suivi rigoureux tout au long du semestre. Ses conseils nous ont permis de maintenir le cap et de structurer notre travail de manière professionnelle.

Nous adressons également nos sincères remerciements aux **élèves ingénieurs du niveau supérieur** ainsi qu'aux **lauréats** qui ont partagé leurs retours d'expérience et leurs recommandations, contribuant ainsi à l'amélioration continue de notre projet.

Nous tenons à exprimer notre reconnaissance aux **membres du jury** pour l'attention qu'ils porteront à l'évaluation de notre travail et pour le temps qu'ils nous accorderont lors de la soutenance.

Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet par leur soutien, leurs encouragements ou leur expertise.

L'équipe CV_Generator

Résumé

Ce projet présente la conception et le développement de **Propel**, une plateforme web intelligente de génération de CV assistée par intelligence artificielle. L'application permet aux utilisateurs de générer un curriculum vitae personnalisé et optimisé pour une offre d'emploi spécifique en quelques secondes. Le système repose sur une architecture en microservices combinant Angular pour le frontend, ASP.NET Core pour les services backend, et une pipeline d'agents IA en Python utilisant LangChain et LangGraph pour l'extraction, l'analyse, l'optimisation et la livraison des CV. L'infrastructure est entièrement conteneurisée avec Docker et déployée via un pipeline CI/CD complet intégrant des outils de sécurité et de qualité (SonarCloud, Trivy, OWASP ZAP). Le projet a été réalisé par une équipe de six développeurs fullstack selon une méthodologie SCRUM adaptée.

Mots-clés : Génération de CV, Intelligence Artificielle, Microservices, Angular, ASP.NET Core, LangChain, Docker, DevSecOps.

Abstract

This project presents the design and development of **Propel**, an intelligent web platform for AI-powered CV generation. The application enables users to generate a tailored and optimized curriculum vitae for a specific job offer in seconds. The system is built on a microservices architecture combining Angular for the frontend, ASP.NET Core for backend services, and a Python AI agent pipeline using LangChain and LangGraph for extraction, analysis, optimization, and delivery of CVs. The infrastructure is fully containerized with Docker and deployed through a complete CI/CD pipeline integrating security and quality tools (SonarCloud, Trivy, OWASP ZAP). The project was carried out by a team of six full-stack developers following an adapted SCRUM methodology.

Keywords : CV Generation, Artificial Intelligence, Microservices, Angular, ASP.NET Core, LangChain, Docker, DevSecOps.

Liste des acronymes

.NET Microsoft .NET Framework. [iv](#)

Angular Angular Framework. [iv](#)

API Application Programming Interface. [iv](#)

ATS Applicant Tracking System. [iv](#)

CD Continuous Deployment. [iv](#)

CI Continuous Integration. [iv](#)

CRUD Create, Read, Update, Delete. [iv](#)

DAST Dynamic Application Security Testing. [iv](#)

E2E End-to-End. [iv](#)

gRPC gRPC Remote Procedure Call. [iv](#)

HTTP HyperText Transfer Protocol. [iv](#)

HTTPS HyperText Transfer Protocol Secure. [iv](#)

IaC Infrastructure as Code. [iv](#)

JSON JavaScript Object Notation. [iv](#)

JWT JSON Web Token. [iv](#)

Kafka Apache Kafka. [iv](#)

LLM Large Language Model. [iv](#)

OAuth Open Authorization. [iv](#)

OIDC OpenID Connect. [iv](#)

OWASP Open Web Application Security Project. [iv](#)

PDF Portable Document Format. [iv](#)

PR Pull Request. [iv](#)

RAG Retrieval-Augmented Generation. [iv](#)

REST Representational State Transfer. [iv](#)

S3 Simple Storage Service. [iv](#)

SOLID Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. [iv](#)

SPA Single Page Application. [iv](#)

SQL Structured Query Language. [iv](#)

SSO Single Sign-On. [iv](#)

VM Virtual Machine. [iv](#)

XSS Cross-Site Scripting. [iv](#)

YARP Yet Another Reverse Proxy. [iv](#)

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des acronymes	iv
1 Introduction générale	1
2 Contexte général du projet	2
2.1 Introduction	2
2.2 Présentation du projet	2
2.2.1 Cadre du projet	2
2.2.2 Objectifs du projet	2
2.2.3 Méthodologie de gestion de projet	3
2.2.4 Présentation de l'équipe	3
2.2.5 Planification du projet	5
2.3 Conclusion	5
3 Analyse et conception	6
3.1 Introduction	6
3.2 Étude de l'existant	6
3.3 Étude des besoins fonctionnels	7
3.4 Étude des besoins non fonctionnels	8
3.5 Identification des acteurs	8
3.6 Modélisation du système	9
3.6.1 Diagramme de cas d'utilisation	9
3.6.2 Diagramme de classes	13
3.6.3 Diagramme de séquence — Flux d'authentification	14
3.6.4 Modèle relationnel de la base de données	15
3.7 Conclusion	22

4	Réalisation	23
4.1	Introduction	23
4.2	Étude technique	23
4.2.1	Architecture physique	23
4.2.2	Architecture logicielle	25
4.2.3	Présentation des outils utilisés	26
4.3	Cycle DevSecOps	27
4.3.1	Présentation et importance dans le projet	27
4.3.2	Outils DevSecOps utilisés	27
4.3.3	Application du cycle DevSecOps dans le projet	28
4.3.4	Sécurité applicative	28
4.4	Gestion du code source	28
4.4.1	Utilisation de Git	28
4.4.2	Organisation des branches	29
4.4.3	Gestion des commits et collaboration	29
4.5	Implémentation	29
4.5.1	Développement Backend	29
4.5.2	Pipeline de génération de CV	30
4.5.3	Développement Frontend	31
4.5.4	Base de données	32
4.6	Déploiement	32
4.6.1	Mise en place de l'environnement de développement	32
4.6.2	Dockerisation de l'application	32
4.6.3	Pipeline CI/CD	32
4.7	Présentation de l'application	33
4.7.1	Interface utilisateur	33
4.7.2	Fonctionnalités principales	33
4.7.3	Démonstrations du fonctionnement	33
4.8	Conclusion	33
5	Conclusion générale et perspectives	34
5.1	Conclusion générale	34
5.2	Perspectives	34

Table des figures

2.1	Organigramme fonctionnel de l'équipe	5
2.2	Diagramme de Gantt prévisionnel	5
2.3	Diagramme de Gantt réel	5
3.1	Cas d'utilisation — Authentification	9
3.2	Cas d'utilisation — Gestion du profil	10
3.3	Cas d'utilisation — Génération de CV	11
3.4	Cas d'utilisation — Suivi des candidatures	12
3.5	Cas d'utilisation — Offres d'emploi et notifications	13
3.6	Diagramme de classes — Relations inter-services	13
3.7	Diagramme de séquence — Flux d'authentification OIDC	14
3.8	Modèle relationnel — Base Utilisateurs	16
3.9	Modèle relationnel — Base Contenu Utilisateur	17
3.10	Modèle relationnel — Base Candidatures	18
3.11	Modèle relationnel — Base Workflow	19
3.12	Modèle relationnel — Base Offres d'Emploi	20
3.13	Modèle relationnel — Base CV	21
3.14	Modèle relationnel — Base Notifications	22
4.1	Architecture de déploiement locale — Répartition des VMs par membre	24
4.2	Architecture logicielle du système	25
4.3	Organisation des branches Git	29
4.4	Flux des événements Kafka — Publications et consommations	30
4.5	Pipeline de génération de CV — De l'offre d'emploi à l'envoi par email	31
4.6	Pipeline CI/CD	32
4.7	Interface du tableau de bord	33
4.8	Interface de génération de CV	33
4.9	Tableau Kanban de suivi des candidatures	33
4.10	Interface de gestion du portfolio	33

Liste des tableaux

2.1	Répartition des rôles au sein de l'équipe	4
3.1	Comparaison des solutions existantes	6
3.2	Besoins fonctionnels du système	7
3.3	Identification des acteurs du système	8
4.1	Outils de développement	26
4.2	Outils CI/CD et qualité	27

Chapitre 1

Introduction générale

Dans un monde professionnel de plus en plus compétitif, la qualité et la pertinence d'un curriculum vitae (CV) jouent un rôle déterminant dans le succès d'une candidature. Chaque offre d'emploi possède ses propres exigences, et un CV générique a souvent moins d'impact qu'un CV adapté spécifiquement au poste visé. Cependant, personnaliser manuellement son CV pour chaque candidature est un processus chronophage et fastidieux.

C'est dans ce contexte que s'inscrit le projet **Propel**, une plateforme web intelligente de génération de CV assistée par intelligence artificielle. L'objectif est de permettre aux candidats de générer un CV personnalisé et optimisé pour une offre d'emploi donnée en quelques secondes, tout en assurant une compatibilité optimale avec les systèmes de suivi des candidatures (ATS).

Ce rapport présente l'ensemble du travail réalisé par notre équipe dans le cadre du projet d'intégration de la 1ère année Génie Informatique, sous la supervision de M. GHAILANI.

Le rapport est structuré en trois chapitres principaux. Le premier chapitre pose le contexte général du projet, présente les objectifs, la méthodologie de gestion adoptée ainsi que l'équipe de réalisation. Le deuxième chapitre est consacré à l'analyse et la conception du système, incluant l'étude des besoins, l'identification des acteurs et la modélisation. Enfin, le troisième chapitre détaille la réalisation technique du projet, couvrant l'architecture, les outils utilisés, le cycle DevSecOps, l'implémentation et le déploiement.

Chapitre 2

Contexte général du projet

2.1 Introduction

Ce premier chapitre présente le cadre général du projet **Propel**. Nous y décrivons le contexte dans lequel s’inscrit le projet, les objectifs visés, la méthodologie de gestion adoptée, ainsi que la composition et l’organisation de l’équipe de réalisation.

2.2 Présentation du projet

2.2.1 Cadre du projet

Ce projet est réalisé dans le cadre du module **Projet d’Intégration** de la 1ère année Génie Informatique, sous la supervision de M. GHAILANI. Il constitue une mise en situation professionnelle où les étudiants sont amenés à concevoir, développer et déployer une application complète en appliquant les compétences acquises durant leur formation.

L’équipe est composée de six étudiants, chacun jouant le rôle de développeur full-stack avec une responsabilité secondaire spécifique (CI/CD, infrastructure cloud, infrastructure locale, sécurité, qualité, monitoring), reflétant ainsi une organisation DevOps complète.

2.2.2 Objectifs du projet

Les objectifs principaux du projet **Propel** sont les suivants :

- **Génération intelligente de CV** : Permettre à un utilisateur de soumettre une offre d’emploi et de recevoir un CV personnalisé et optimisé pour cette offre, généré automatiquement par une pipeline d’agents d’intelligence artificielle.
- **Suivi des candidatures** : Offrir un tableau de bord complet avec vue Kanban, liste et calendrier pour gérer l’ensemble des candidatures.

- **Gestion de portfolio** : Permettre à l'utilisateur de gérer ses projets, compétences, expériences et formations.
- **Agrégation d'offres d'emploi** : Collecter automatiquement des offres d'emploi depuis différentes plateformes via un agent de crawling.
- **Notifications temps réel** : Notifier l'utilisateur sur l'état d'avancement de ses candidatures et de ses générations de CV.
- **Déploiement industrialisé** : Mettre en place un pipeline CI/CD complet avec conteneurisation, tests automatisés, analyse de qualité et scans de sécurité.

2.2.3 Méthodologie de gestion de projet

Pour la gestion de ce projet, nous avons adopté une approche **SCRUM** adaptée à notre contexte. Le professeur joue le rôle de **Product Owner**, définissant les objectifs et validant les livrables. Le chef d'équipe assume le rôle de **SCRUM Master**, facilitant la coordination et s'assurant de l'avancement des tâches.

La méthodologie mise en œuvre se caractérise par :

- **Réunions quotidiennes** : Des sessions de coordination via **Google Meets** sont organisées régulièrement pour faire le point sur l'avancement, discuter des obstacles et planifier les tâches à venir.
- **Communication continue** : Un groupe **WhatsApp** dédié permet une communication réactive pour la résolution rapide des problèmes et le partage d'information.
- **Organisation en sprints** : Le projet est découpé en itérations (sprints) avec des objectifs clairs à chaque étape, permettant une livraison incrémentale des fonctionnalités.
- **Gestion des tâches** : La répartition des tâches est définie collectivement lors des réunions, en fonction des priorités et des compétences de chaque membre.

Cette approche agile nous a permis de nous adapter aux défis techniques rencontrés tout au long du projet et de maintenir un rythme de travail soutenu.

2.2.4 Présentation de l'équipe

L'équipe **CV_Generator** est composée de six membres, chacun occupant un rôle de développeur fullstack avec une responsabilité transverse spécifique. Cette organisation permet de couvrir l'ensemble du cycle de vie du développement logiciel, du code au déploiement en passant par la qualité et la sécurité.

TABLE 2.1 – Répartition des rôles au sein de l'équipe

Rôle	Eff.	Intervenant	Remarque
Chef d'équipe / Responsable CI/CD	1	Bilal Magguilej	Coordination d'équipe, pipeline CI/CD, Dockerisation complète, validation des PR
Développeur Fullstack – Infra Cloud	1	Mouhssine El Haouary	Déploiement cloud, environnement de production
Développeur Fullstack – Infra Locale	1	Somia Jdia	Terraform, provisionnement VMs locales, environnement de développement
Développeur Fullstack – Sécurité	1	Nourelhouda Ouhba	Keycloak SSO, OWASP ZAP, Trivy, authentification OAuth
Développeur Fullstack – QA	1	Mohssine El Addaoui	Tests E2E Cypress, tests de charge k6, assurance qualité
Développeur Fullstack – Monitoring	1	Salman El-Ghrich	Prometheus, Grafana, observabilité et métriques

Chaque membre a participé activement au développement des trois couches de l'application : frontend (Angular), backend (.NET) et agents IA (Python/FastAPI). Les responsabilités secondaires ont été réparties pour couvrir l'ensemble des aspects DevOps et assurer la qualité et la maintenabilité du projet.

FIGURE 2.1 – Organigramme fonctionnel de l'équipe

2.2.5 Planification du projet

Le projet a été planifié sur la durée du semestre avec des jalons clés correspondant aux livrables attendus. Le diagramme de Gantt ci-dessous présente la planification prévisionnelle ainsi que le déroulement réel du projet.

FIGURE 2.2 – Diagramme de Gantt prévisionnel

FIGURE 2.3 – Diagramme de Gantt réel

2.3 Conclusion

Ce premier chapitre a présenté le cadre général du projet **Propel**, les objectifs visés, la méthodologie de gestion adoptée ainsi que l'organisation de l'équipe. Le chapitre suivant sera consacré à l'analyse détaillée des besoins et à la conception du système.

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre présente l’analyse approfondie du système **Propel**. Nous commençons par une étude des solutions existantes, suivie de l’identification des besoins fonctionnels et non fonctionnels. Ensuite, nous identifions les acteurs du système avant de présenter les différents diagrammes de modélisation.

3.2 Étude de l’existant

Plusieurs solutions de création de CV existent sur le marché. Le tableau suivant présente une comparaison des principales solutions :

TABLE 3.1 – Comparaison des solutions existantes

Solution	CV per-sonnalisé	IA	Suivi can-didatures	Open Source
Propel (notre solution)	Oui	Oui	Oui	Oui
Canva	Partiel	Non	Non	Non
Zety	Modèles fixes	Non	Non	Non
Novoresume	Modèles fixes	Non	Non	Non
Rezi	Oui	Oui	Non	Non
Enhancv	Partiel	Oui	Non	Non

Les solutions existantes présentent plusieurs limitations :

- **Personnalisation limitée** : La plupart des outils offrent des modèles fixes sans adaptation réelle au contenu de l’offre d’emploi.

- **Absence de suivi intégré** : Aucune solution ne combine la génération de CV avec le suivi des candidatures dans une plateforme unifiée.
- **Coût** : Les solutions avancées sont payantes (abonnements mensuels).
- **Optimisation ATS limitée** : Peu d'outils optimisent réellement le CV pour passer les filtres des systèmes ATS.

Propel se distingue en offrant une solution complète, gratuite et open source, combinant génération IA de CV, optimisation ATS et suivi de candidatures.

3.3 Étude des besoins fonctionnels

Les besoins fonctionnels identifiés pour le système **Propel** sont les suivants :

TABLE 3.2 – Besoins fonctionnels du système

ID	Description
BF01	L'utilisateur doit pouvoir s'authentifier via Keycloak (email, Google, GitHub)
BF02	L'utilisateur doit pouvoir gérer son portfolio (projets, compétences, expériences, formations)
BF03	L'utilisateur doit pouvoir soumettre une offre d'emploi pour générer un CV personnalisé
BF04	Le système doit extraire les informations clés de l'offre d'emploi via un agent IA
BF05	Le système doit rechercher les compétences pertinentes dans le profil de l'utilisateur via RAG
BF06	Le système doit générer un CV formaté selon un template optimisé ATS
BF07	Le système doit optimiser le contenu du CV pour maximiser la compatibilité ATS
BF08	Le système doit envoyer le CV généré par email à l'utilisateur
BF09	L'utilisateur doit pouvoir suivre ses candidatures (vue Kanban, liste, calendrier)
BF10	L'utilisateur doit pouvoir visualiser des statistiques et des graphiques sur ses candidatures
BF11	Le système doit crawler automatiquement des offres d'emploi depuis des plateformes externes
BF12	L'utilisateur doit recevoir des notifications temps réel sur l'état de ses candidatures
BF13	L'utilisateur doit pouvoir gérer plusieurs versions de CV
BF14	Le système doit exporter les CV au format PDF

3.4 Étude des besoins non fonctionnels

Les besoins non fonctionnels identifiés sont :

- **Performance** : La génération d'un CV ne doit pas excéder quelques secondes. Les temps de réponse de l'API doivent être optimisés.
- **Sécurité** : L'authentification doit être centralisée via Keycloak avec support OAuth2 et OpenID Connect. Les communications doivent être chiffrées (HTTPS). Les vulnérabilités doivent être scannées via Trivy et OWASP ZAP.
- **Disponibilité** : Le système doit être résilient grâce à une architecture en microservices et une orchestration Docker. Les services critiques doivent pouvoir redémarrer automatiquement.
- **Maintenabilité** : Le code source doit être organisé en modules séparés, versionné avec Git, et suivre les principes SOLID. La qualité du code est surveillée via SonarCloud.
- **Évolutivité** : L'architecture microservices permet de faire évoluer chaque service indépendamment. L'utilisation de Kafka permet un couplage faible entre les services.
- **Ergonomie** : L'interface utilisateur doit être intuitive, responsive et suivre une charte graphique cohérente (design system Propel).
- **Portabilité** : L'ensemble de l'application doit être conteneurisée avec Docker pour garantir un déploiement reproductible sur n'importe quel environnement.

3.5 Identification des acteurs

L'analyse du système **Propel** permet d'identifier les acteurs suivants :

TABLE 3.3 – Identification des acteurs du système

Acteur	Description
Utilisateur	Candidat cherchant à générer des CV personnalisés et suivre ses candidatures
Système d'authentification	Keycloak gère l'authentification et l'autorisation via OAuth2/OpenID Connect
Pipeline IA	Ensemble d'agents intelligents qui traitent les offres d'emploi et génèrent les CV
Système de messagerie	Kafka assure la communication asynchrone entre les microservices
Administrateur	Gère la configuration du système et supervise le déploiement

3.6 Modélisation du système

3.6.1 Diagramme de cas d'utilisation

Les diagrammes de cas d'utilisation ci-dessous présentent les interactions entre l'utilisateur et les différentes fonctionnalités du système. Cinq diagrammes couvrent les principaux modules de l'application.

Authentification L'utilisateur peut créer un compte, se connecter et se déconnecter du système.

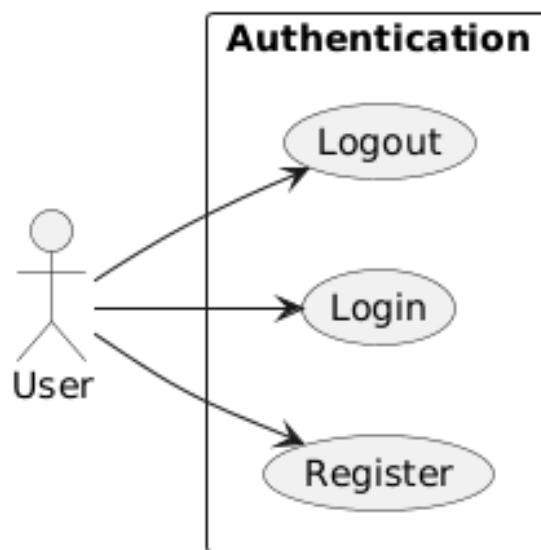


FIGURE 3.1 – Cas d'utilisation — Authentification

Gestion du profil L'utilisateur peut gérer l'ensemble de ses données personnelles et professionnelles : informations de profil, projets, compétences, expériences, formations, certifications, langues, liens sociaux et profils CV. Tous ces cas nécessitent une authentification préalable.

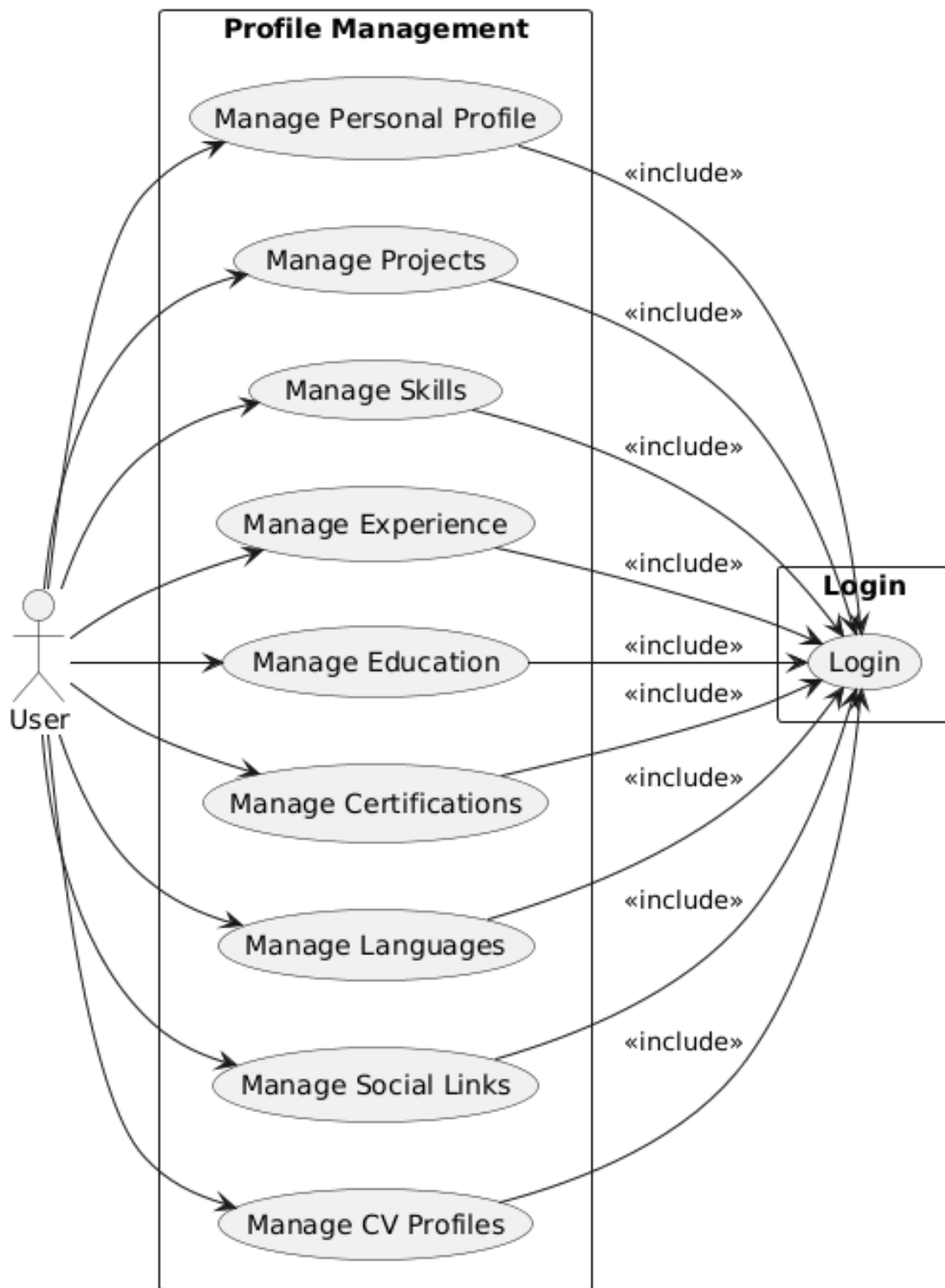


FIGURE 3.2 – Cas d'utilisation — Gestion du profil

Génération de CV L'utilisateur peut générer un CV personnalisé à partir d'une offre d'emploi, choisir un template, définir le ton du contenu, exporter le CV au format PDF et consulter l'historique des générations. Le système d'IA intervient comme acteur

secondaire dans le processus de génération.

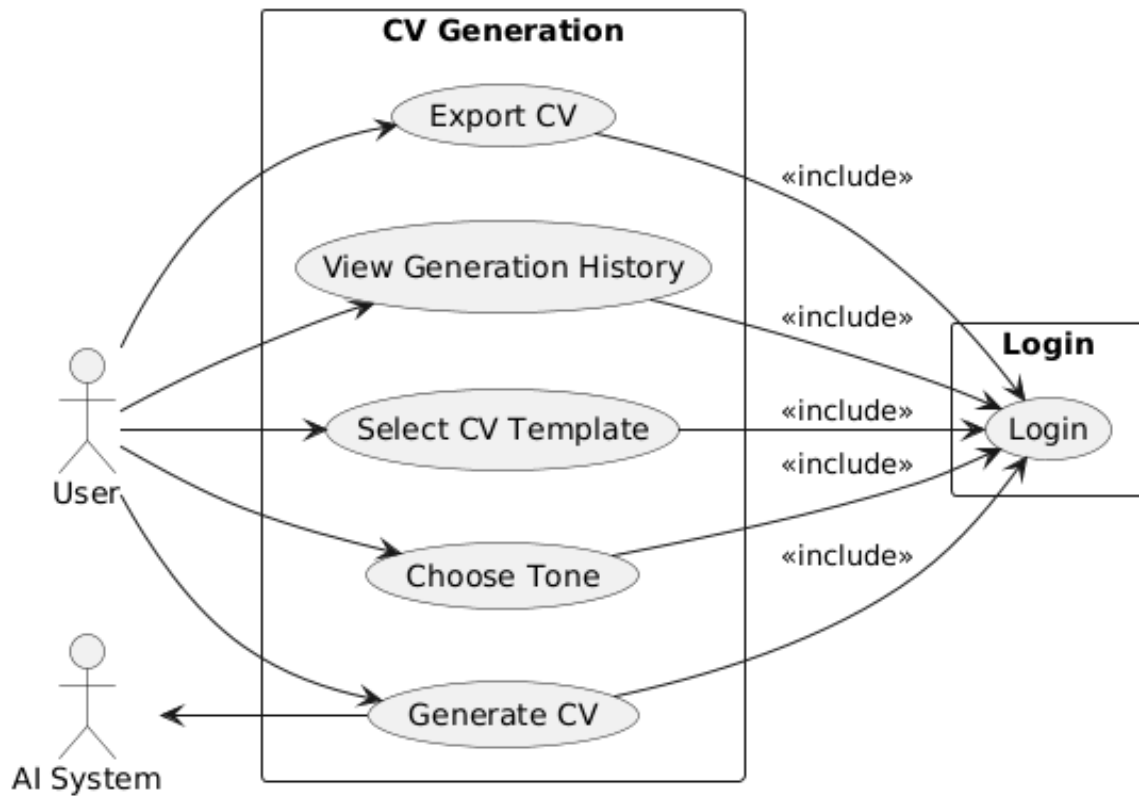


FIGURE 3.3 – Cas d'utilisation — Génération de CV

Suivi des candidatures L'utilisateur peut ajouter une candidature, mettre à jour son statut, visualiser le tableau Kanban, consulter les analyses statistiques et le calendrier des événements.

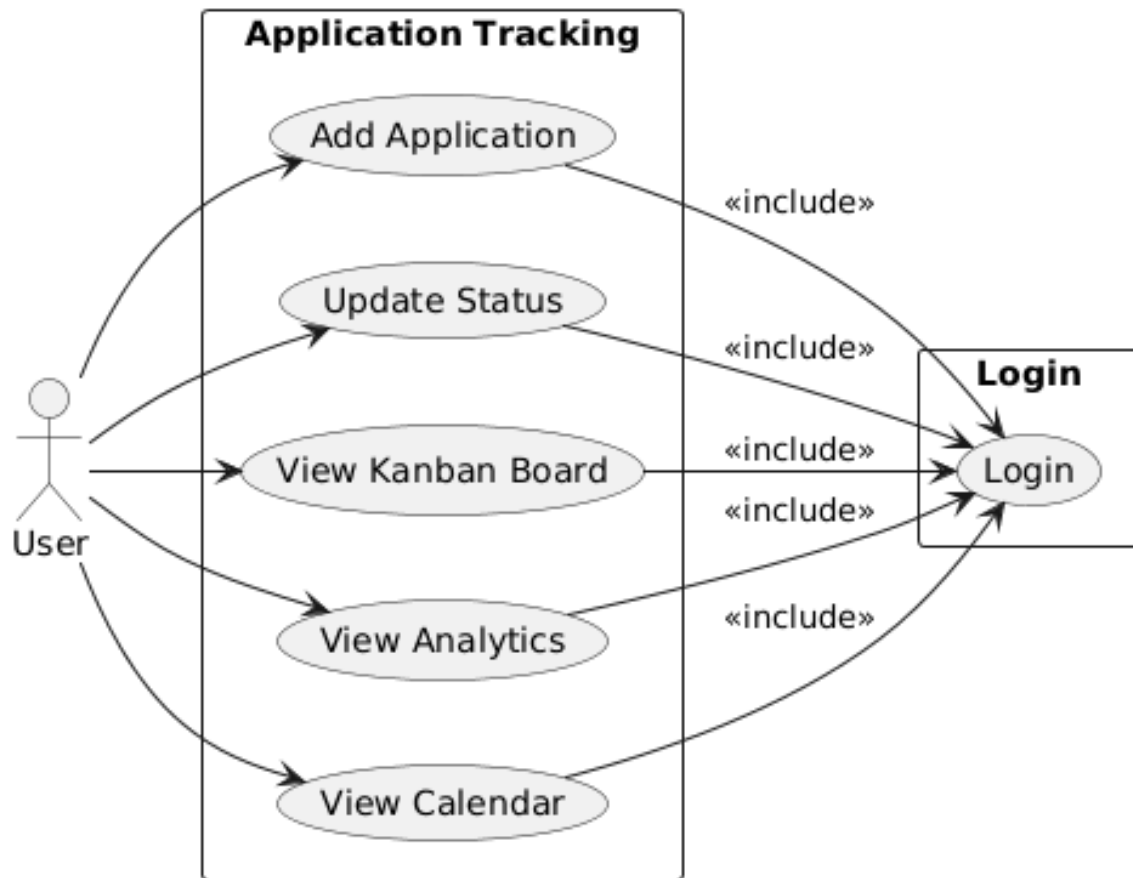


FIGURE 3.4 – Cas d'utilisation — Suivi des candidatures

Offres d'emploi et notifications L'utilisateur peut parcourir les offres d'emploi, déclencher un crawling en direct, configurer ses notifications, planifier des rappels et programmer des envois d'emails.

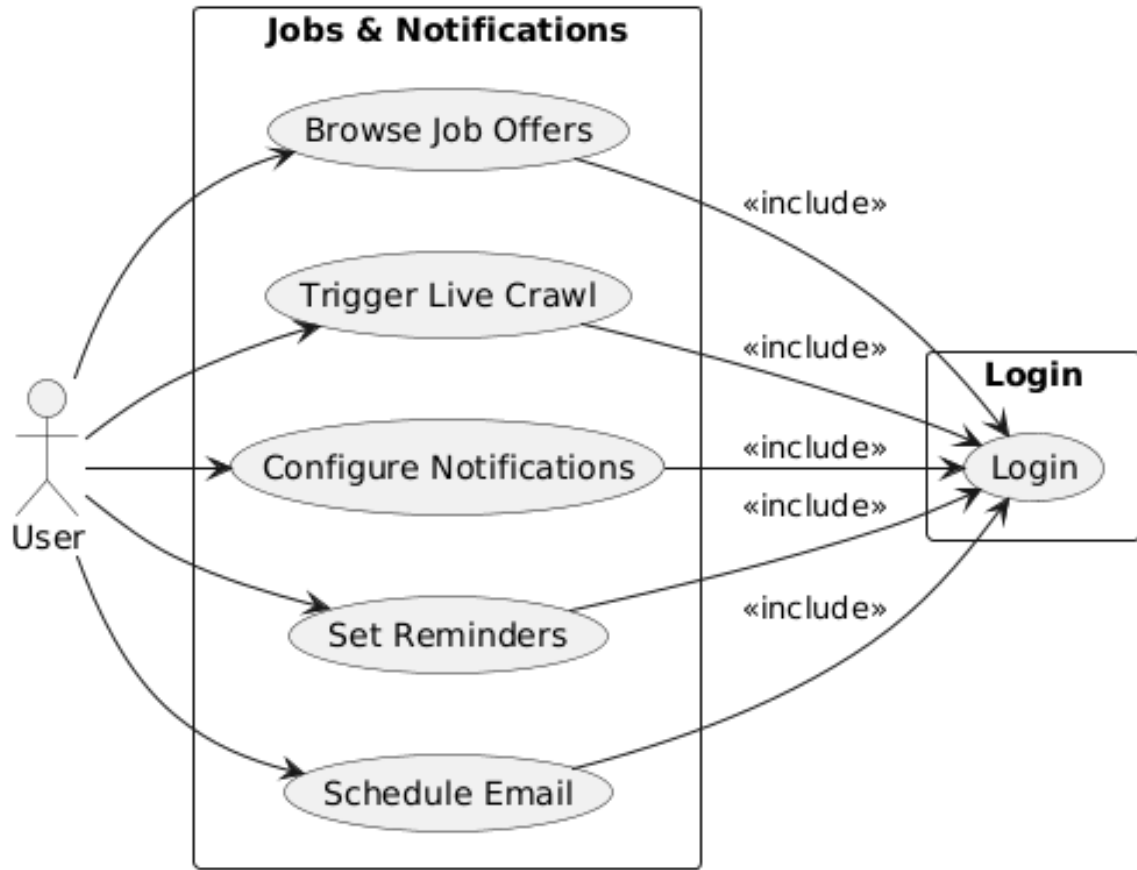


FIGURE 3.5 – Cas d’utilisation — Offres d’emploi et notifications

3.6.2 Diagramme de classes

Le diagramme de classes ci-dessous illustre les relations entre les entités principales du système, organisées par base de données. Chaque package coloré représente un microservice et sa base de données associée. Les relations montrent les dépendances inter-services, comme les références de l'utilisateur vers les contenus, candidatures et CV, ou les liens entre une candidature et une version de CV.

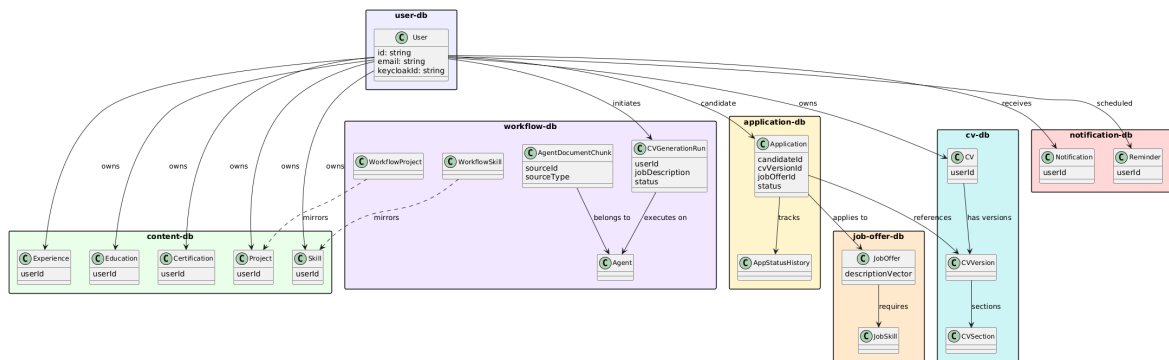


FIGURE 3.6 – Diagramme de classes — Relations inter-services

3.6.3 Diagramme de séquence — Flux d'authentification

Le diagramme de séquence ci-dessous illustre le flux d'authentification via Keycloak avec le protocole OpenID Connect (OIDC). L'utilisateur est redirigé vers Keycloak pour s'authentifier, reçoit un code d'autorisation, puis l'API Gateway échange ce code contre des jetons JWT. Un cookie de session (`cv_session`) est créé pour maintenir la session. Lors de l'inscription, un événement `user.registered` est publié sur Kafka pour notifier les autres services.

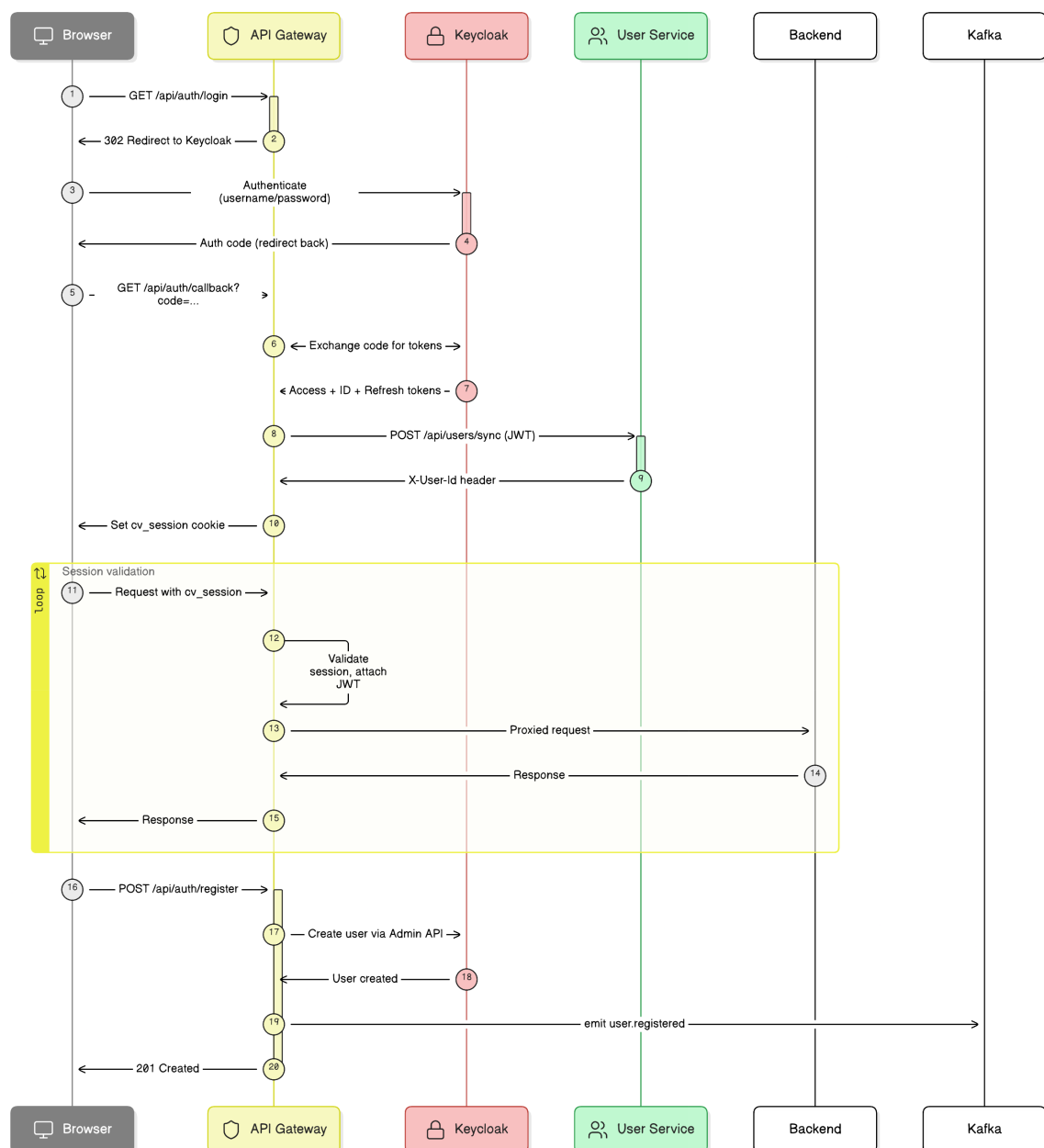


FIGURE 3.7 – Diagramme de séquence — Flux d'authentification OIDC

3.6.4 Modèle relationnel de la base de données

L'architecture microservices du projet implique l'utilisation de plusieurs bases de données PostgreSQL 16, une par service. Chaque base de données est spécialisée pour les besoins de son service et réside dans un conteneur Docker dédié. L'extension pg-vector est utilisée pour le support des recherches vectorielles dans les bases concernées par le pipeline IA.

Base Utilisateurs (user-db — port 5433)

La base utilisateurs centralise les informations personnelles et professionnelles des candidats.

- **users** : Table principale contenant le profil complet de chaque utilisateur. Les attributs incluent **keycloakId** (identifiant de synchronisation avec Keycloak), **headline** (titre professionnel), **desiredSalaryMin/desiredSalaryMax** (prétentions salariales), **requiresVisaSponsorship**, **noticePeriod** (préavis), **employmentTypes** (types d'emploi souhaités), **remotePreference** (préférence de télétravail), **willingToRelocate** (mobilité géographique). Des colonnes JSON permettent de stocker des données flexibles : **preferencesJson** et **aiProfileDataJson** (profil généré par l'IA).

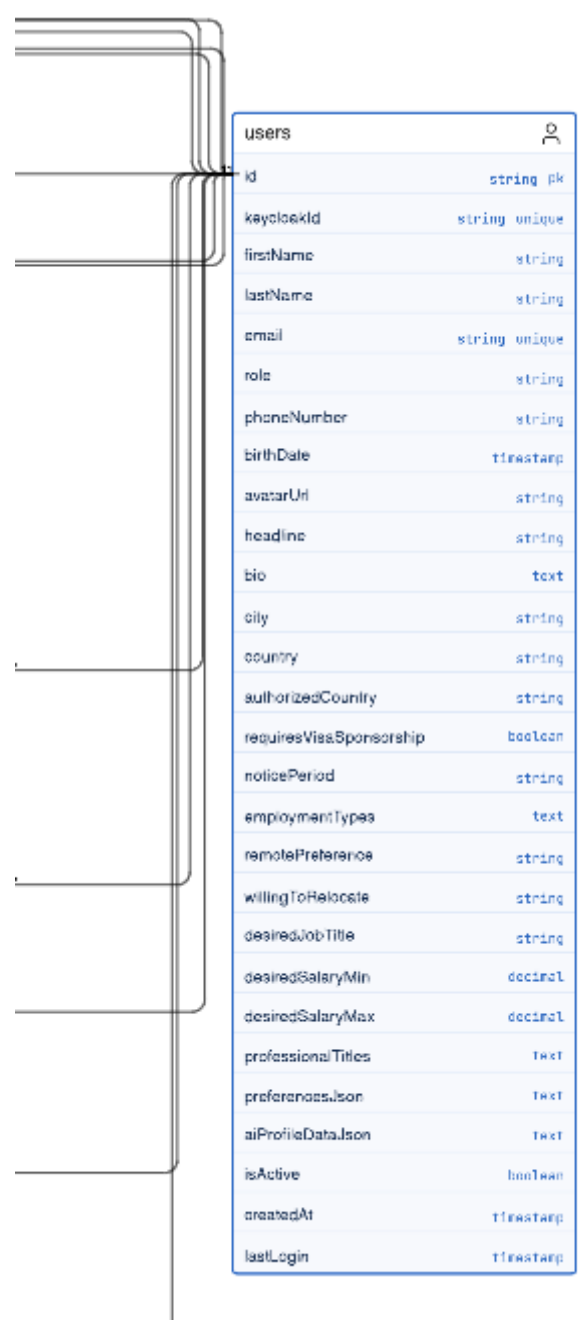


FIGURE 3.8 – Modèle relationnel — Base Utilisateurs

Base Contenu Utilisateur (content-db — port 5434)

Cette base gère l'ensemble du portfolio de l'utilisateur. Chaque table est reliée à l'utilisateur via une clé étrangère `userId`.

- **projects** : Projets réalisés. L'attribut `skillsJson` stocke les compétences associées sous forme JSON. `repositoryUrl` et `demoUrl` pointent vers les sources et démonstrations.
- **skills** : Compétences avec `category` (langage, framework, outil), `level` (débutant, intermédiaire, avancé, expert) et `yearsOfExperience`.

- **experiences** : Expériences professionnelles avec **company** et **status** (en cours, terminée).
- **educations** : Formations avec **degreeType** (licence, master, doctorat), **fieldOfStudy** et **specialization**. **diplomaFileUrl** stocke le lien vers le fichier du diplôme.
- **certifications** : Certifications avec **issuingOrganization** et **credentialUrl**.
- **languages** : Langues avec **level** (A1 à C2).
- **interests** : Centres d'intérêt.
- **hackathons** : Participations à des hackathons avec **role**, **result** et **description**.
- **socialLinks** : Liens vers les réseaux sociaux (**platform**, **url**).
- **cvProfiles** : Profils de CV avec **summary** (résumé personnel) et **title**.
- **academicActivities** : Activités académiques (projets scolaires, publications) avec période.



FIGURE 3.9 – Modèle relationnel — Base Contenu Utilisateur

Base Candidatures (application-db — port 5436)

Cette base assure le suivi des candidatures des utilisateurs.

- **applications** : Candidatures avec **candidateId**, **cvVersionId** et **jobOfferId** comme clés étrangères. **companyName** et **positionTitle** décrivent le poste visé.

`status` peut prendre les valeurs : `saved`, `applied`, `interview`, `offer`, `rejected`. `offerSource` indique la provenance de l'offre. `isSaved` permet de marquer une candidature comme favorite.

- `applicationStatusHistory` : Historique des changements de statut. `oldStatus` et `newStatus` tracent chaque transition. `changedBy` enregistre l'acteur (utilisateur ou système) et `comment` permet d'ajouter une note.
- `applicationConfigurations` : Préférences utilisateur pour le module. `showReminders` active les rappels, `selectedStatuses` filtre les statuts affichés.

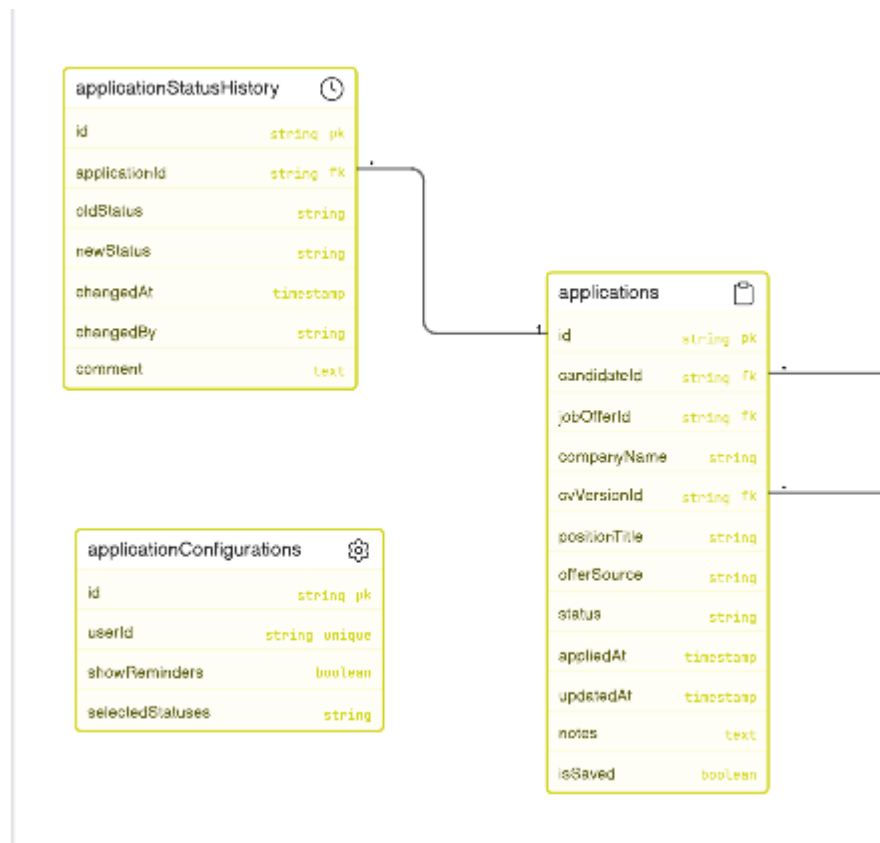


FIGURE 3.10 – Modèle relationnel — Base Candidatures

Base Workflow (workflow-db — port 5435)

La base workflow est la plus complexe. Elle pilote le pipeline de génération de CV et utilise pgvector pour les recherches vectorielles.

- `workflows` : Définition des pipelines. `definitionJson` contient la configuration du workflow au format JSON. `isActive` active ou désactive le pipeline.
- `cvGenerationRuns` : Traces d'exécution de chaque génération de CV. `jobDescription` stocke le texte brut de l'offre. `status` suit l'état (`pending`, `running`, `completed`, `failed`, `cancelled`). `stepStatuses` est un champ JSON contenant l'état de chaque étape. Les résultats des agents sont stockés dans des colonnes dédiées : `extractionResult`, `searchResult`, `renderResult`, `optimizationResult`, `deliveryResult`.

- **agents** : Configuration des agents IA. **agentId** est un identifiant unique (ex : job-extractor). **sortOrder** détermine l'ordre d'exécution dans le pipeline. **backgroundGradient** est utilisé pour l'affichage dans l'interface.
- **agentDocumentChunks** : Stockage des documents vectorisés pour le RAG. **sourceId** et **sourceType** identifient l'entité source. **embedding** est un vecteur pgvector utilisé pour la recherche sémantique. **searchVector** est un tsvector pour la recherche full-text.
- **jobExtractions** : Extractions d'offres d'emploi. **inputType** distingue le texte saisi de l'URL. **confidence** est un score de confiance de l'extraction. **outputJson** contient le résultat structuré.
- **workflowProjects**, **workflowSkills**, **workflowExperiences** : Entités miroir du contenu utilisateur, optimisées pour la recherche vectorielle avec des champs **embedding** et **aiSummaryJson**.



FIGURE 3.11 – Modèle relationnel — Base Workflow

Base Offres d'Emploi (job-offer-db — port 5437)

Cette base stocke les offres d'emploi collectées par l'agent de crawling et enrichies par l'agent d'extraction.

- **jobOffers** : Offres d'emploi. **jobHash** est un hash unique de l'offre pour éviter les doublons lors du crawling. **sourceUrl** est l'URL d'origine. **rawDescription** stocke la description brute. **descriptionVector** est un vecteur pgvector pour le matching avec le profil utilisateur. **locationType** distingue **remote**, **onsite** et **hybrid**. **employmentType** (**full-time**, **part-time**, **contract**) et **seniorityLevel**

(junior, mid, senior, lead) qualifient le poste.

- **jobSkills** : Compétences requises pour le poste. **type** distingue les compétences **required** et **preferred**. **isMandatory** indique si la compétence est obligatoire.
- **jobResponsibilities** : Liste des responsabilités du poste.
- **jobBenefits** : Avantages proposés par l'entreprise.
- **searchCaches** : Cache des recherches de crawling. **keyword** et **location** définissent la recherche. **status** suit l'avancement. **expectedCount** et **processedCount** permettent de suivre le nombre d'offres traitées.
- **userQuotas** : Quotas de crawling par utilisateur. **lastCrawlDate** enregistre la dernière date d'utilisation.

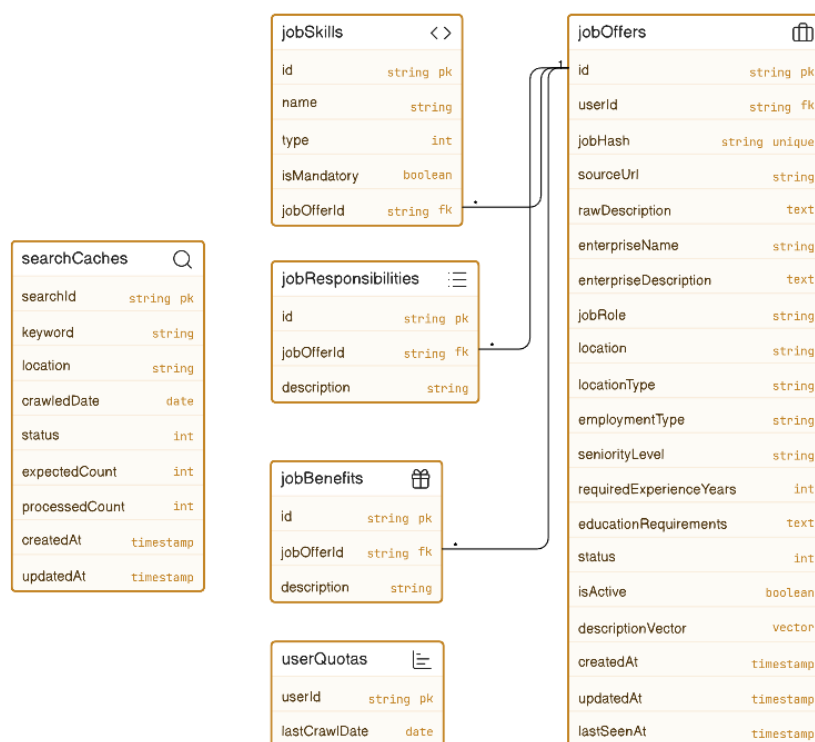


FIGURE 3.12 – Modèle relationnel — Base Offres d'Emploi

Base CV (cv-db — port 5439)

La base CV gère les curriculum vitae et leurs versions successives.

- **cvs** : CV avec **templateId** (identifiant du template utilisé) et **isActive** (CV actif par défaut).
- **cvVersions** : Versions d'un CV. **versionNumber** est un numéro incrémental. **contentJson** stocke le contenu complet du CV au format JSON structuré. **fileUrl** et **pdfUrl** pointent respectivement vers le fichier source et le PDF généré stockés dans MinIO.
- **cvSections** : Sections d'une version de CV. **sectionType** définit le type (**education**,

experience, skills, projects, certifications, languages). `contentJson` contient les données de la section. `displayOrder` détermine l'ordre d'affichage.

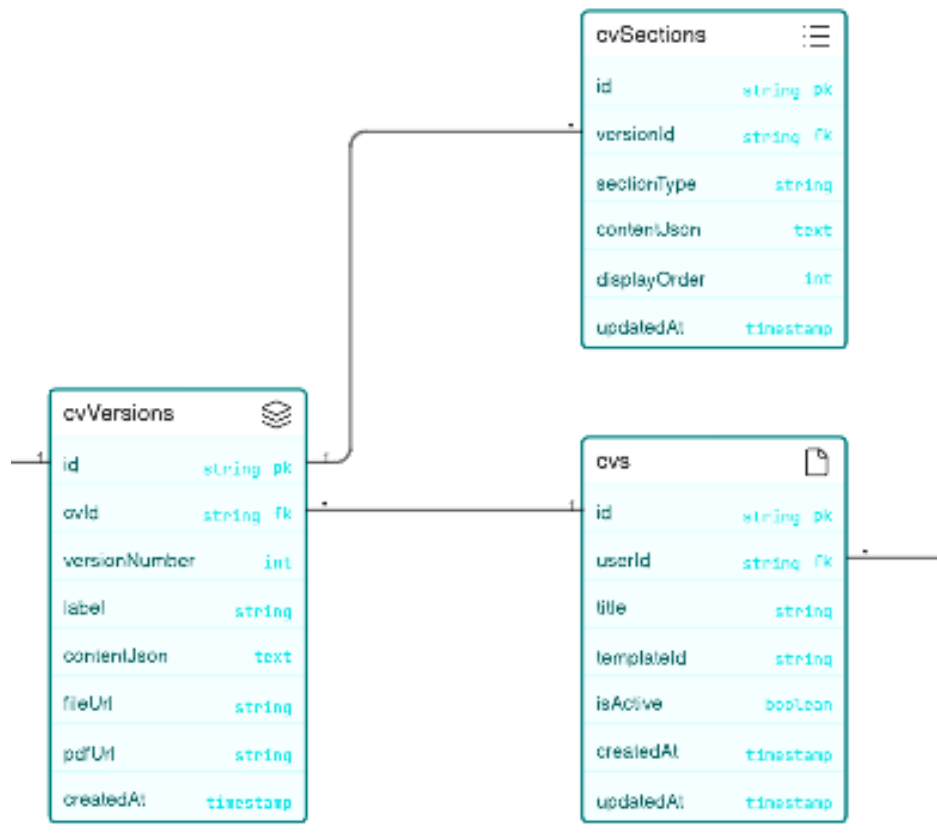


FIGURE 3.13 – Modèle relationnel — Base CV

Base Notifications (notification-db — port 5438)

Cette base gère l'ensemble du système de notifications, de rappels et de messagerie.

- **notifications** : Notifications in-app et email. `type` définit la catégorie (`application_update`, `cv_ready`, `reminder`, `welcome`). `channel` détermine le mode (`email`, `in-app`). `isRead` et `isSent` tracent l'état. `errorMessage` enregistre les échecs d'envoi.
- **notificationPreferences** : Préférences de notification par utilisateur. `enableEmail` et `enableInApp` activent les canaux. `applicationUpdates`, `cvUpdates`, `reminders`, `weeklyDigest` activent les catégories. `defaultReminderDaysBefore` définit le délai de rappel par défaut.
- **reminders** : Rappels planifiés. `eventDate` est la date de l'événement, `reminderAt` est la date du rappel. `reminderOffset` définit le décalage (ex : 1d, 2h). `status` suit l'état (`pending`, `sent`, `cancelled`).
- **contacts** : Carnet d'adresses. `source` indique la provenance (`manuel`, `import Gmail`). `isFavorite` permet de marquer les contacts favoris. `avatarBase64`

stocke l’avatar encodé.

- **emailMessages** : Historique des emails envoyés. **provider** indique le fournisseur (MailKit, Gmail API). **status** suit la délivrabilité. **contactId** relie l’email à un contact. **scheduleId** permet de lier à une campagne planifiée.
- **emailSchedules** : Campagnes d’emails planifiées. **cronExpression** définit la récurrence. **recipientIds** stocke les destinataires. **isActive** active ou désactive la campagne.
- **gmailConnections** : Connexions Gmail des utilisateurs. **encryptedAccessToken** et **encryptedRefreshToken** stockent les jetons OAuth chiffrés. **isRevoked** indique si la connexion a été révoquée.



FIGURE 3.14 – Modèle relationnel — Base Notifications

3.7 Conclusion

Ce chapitre a présenté l’analyse complète du système **Propel**, incluant l’étude de l’existant, les besoins fonctionnels et non fonctionnels, l’identification des acteurs et la modélisation. Le chapitre suivant détaillera la réalisation technique du projet.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la réalisation technique du projet **Propel**. Nous détaillons l'architecture physique et logicielle, les outils utilisés, le cycle DevSecOps mis en place, la gestion du code source, l'implémentation des différentes couches de l'application, ainsi que le déploiement.

4.2 Étude technique

4.2.1 Architecture physique

Infrastructure matérielle

Le projet s'appuie sur une infrastructure locale déployée sur des machines virtuelles VMware Workstation, provisionnées via Terraform. Chaque membre de l'équipe est responsable de VMs dédiées selon la répartition des services.

- **ai_agents/docker-compose.yml** : Contient le pipeline de 6 agents IA en Python avec leur orchestrateur.
- **api_gateway/docker-compose.yml** : Contient la passerelle API (YARP) et Keycloak.

Tous les services partagent un réseau Docker commun (**cv-network**) pour la communication inter-services.

4.2.2 Architecture logicielle

L'architecture logicielle de **Propel** est une architecture en **microservices** avec communication événementielle. Le schéma ci-dessous illustre l'architecture globale du système.

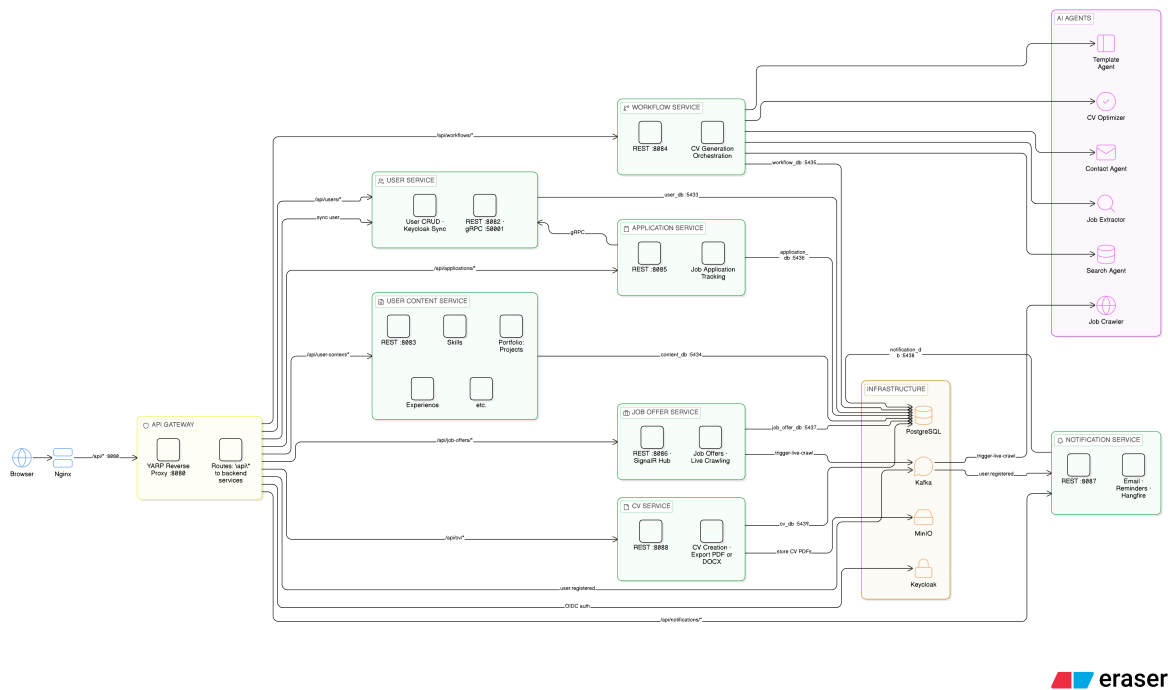


FIGURE 4.2 – Architecture logicielle du système

L'architecture se compose des couches suivantes :

- **Frontend (Angular)** : Application monopage (SPA) hébergée par Nginx, consommant l'API via la passerelle.
- **Passerelle API (YARP)** : Proxy inverse .NET assurant le routage, l'authentification et la gestion des sessions. Publie les événements de connexion vers Kafka.
- **Backend (.NET)** : 7 microservices indépendants, chacun avec sa propre base de données PostgreSQL. Ils communiquent via HTTP, gRPC et Kafka.

- **Agents IA (Python)** : Pipeline de 6 agents FastAPI utilisant LangChain et LangGraph pour le traitement intelligent des CV.
- **Messagerie (Kafka)** : Système de messagerie asynchrone en mode KRaft (sans Zookeeper) pour la communication événementielle entre services.
- **Stockage objet (MinIO)** : Stockage S3-compatible pour les fichiers PDF générés.
- **Authentification (Keycloak)** : Serveur d'authentification centralisé avec support OAuth2, OpenID Connect et fédération d'identités (Google, GitHub).

4.2.3 Présentation des outils utilisés

Outils de développement

Le tableau suivant récapitule les principaux outils et technologies utilisés pour le développement :

TABLE 4.1 – Outils de développement

Couche	Technologies
Frontend	Angular 21, TypeScript, RxJS, Signals, SCSS, Vitest, Cypress
Backend	ASP.NET Core 10, Entity Framework Core, gRPC, YARP, Hangfire, MailKit
Agents IA	Python, FastAPI, LangChain, LangGraph, LangChain-Groq, Pydantic
Bases de données	PostgreSQL 16 avec extension pgvector
Authentification	Keycloak 26, OpenID Connect, JWT
Messagerie	Apache Kafka (KRaft mode)
Stockage	MinIO (API S3)
Conteneurisation	Docker, Docker Compose

Outils d'intégration et de déploiement continu (CI/CD)

TABLE 4.2 – Outils CI/CD et qualité

Outil	Rôle
GitHub Actions	Pipeline CI/CD (6 phases)
SonarCloud	Analyse statique de code et quality gates
Trivy	Scan de vulnérabilités (système de fichiers et images Docker)
OWASP ZAP	Tests de sécurité DAST (Dynamic Application Security Testing)
Docker	Conteneurisation multi-stage
GitHub Container Registry	Registre d'images Docker
DockerHub	Registre d'images secondaire

4.3 Cycle DevSecOps

4.3.1 Présentation et importance dans le projet

Le cycle DevSecOps intègre la sécurité à chaque étape du cycle de développement logiciel. Dans le cadre de **Propel**, cette approche est essentielle étant donné la nature sensible des données traitées (informations personnelles des utilisateurs, CV, données de connexion).

L'adoption du cycle DevSecOps nous permet de :

- Détecter les vulnérabilités tôt dans le cycle de développement.
- Automatiser les contrôles de sécurité.
- Garantir la conformité aux bonnes pratiques.
- Assurer une traçabilité complète des modifications.

4.3.2 Outils DevSecOps utilisés

- **SonarCloud** : Analyse statique de code avec quality gates pour les projets Angular, .NET et Python. Des seuils de qualité (couverture de code, dette technique, duplication) doivent être respectés.
- **Trivy** : Scan de vulnérabilités sur le système de fichiers lors des phases statiques et sur les images Docker construites. Détection des dépendances obsolètes et des failles de sécurité connues.
- **OWASP ZAP** : Tests DAST (Dynamic Application Security Testing) en environnement d'intégration pour détecter les vulnérabilités applicatives (XSS, injection SQL, etc.).

4.3.3 Application du cycle DevSecOps dans le projet

Le cycle DevSecOps est appliqué à travers le pipeline CI/CD en six phases :

1. **Phase 1 - Static Checks** : Validation des fichiers Docker Compose, scan Trivy filesystem, analyse SonarCloud pour le frontend, le backend et les agents IA.
2. **Phase 2 - Component Tests** : Exécution des tests unitaires (.NET avec xUnit, Angular avec Vitest, Python avec pytest) et génération des rapports de couverture.
3. **Phase 3 - Integration / E2E** : Tests d'intégration et tests E2E avec Cypress, suivis d'un scan de sécurité OWASP ZAP.
4. **Phase 4 - Build & Scan** : Construction des images Docker multi-stage, suivie d'un scan Trivy des images.
5. **Phase 5 - Registry Push** : Publication des images validées vers GitHub Container Registry et DockerHub.
6. **Phase 6 - Deploy** : Déploiement de l'application sur l'environnement cible.

4.3.4 Sécurité applicative

La sécurité applicative est assurée à plusieurs niveaux :

- **Authentification centralisée** : Keycloak avec OpenID Connect, support OAuth2 (Google, GitHub), gestion des sessions par cookies sécurisés JWT.
- **Protection des API** : Validation des tokens JWT, gestion des rôles et permissions.
- **Sécurisation des communications** : HTTPS, chiffrement des données sensibles.
- **Scan de vulnérabilités** : Automatisé via Trivy (images et dépendances) et OWASP ZAP (tests dynamiques).
- **Gestion des secrets** : Fichier `.env` avec variables d'environnement, exclusion des secrets du versionnement via `.gitignore`.

4.4 Gestion du code source

4.4.1 Utilisation de Git

Le code source du projet est versionné avec **Git** et hébergé sur **GitHub**. L'utilisation de Git permet un suivi précis de l'historique des modifications, une collaboration efficace entre les membres de l'équipe et une intégration fluide avec le pipeline CI/CD.

4.4.2 Organisation des branches

La stratégie de branches adoptée est une approche **Git Flow** simplifiée :

- **main** : Branche de production, protégée, ne reçoit que des merges validés.
- **develop** : Branche d'intégration continue pour le développement.
- **feature/*** : Branches de fonctionnalités créées à partir de **develop** et fusionnées via Pull Request.
- **hotfix/*** : Branches de correction urgente créées à partir de **main**.

FIGURE 4.3 – Organisation des branches Git

4.4.3 Gestion des commits et collaboration

La collaboration s'effectue via **Pull Requests** sur GitHub. Chaque PR est soumise à une revue de code par le chef d'équipe (Bilal Magguilej) avant d'être fusionnée. Le pipeline CI/CD s'exécute automatiquement sur chaque PR pour valider la qualité et la sécurité du code proposé.

4.5 Implémentation

4.5.1 Développement Backend

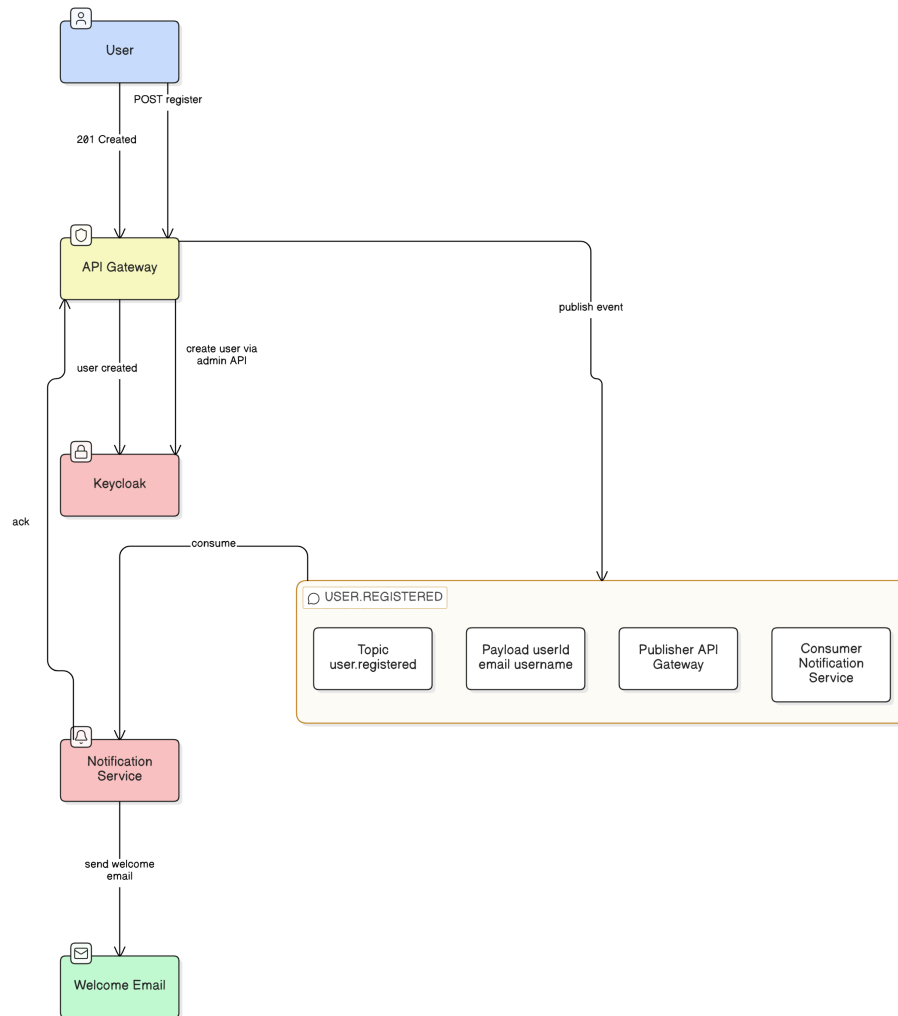
Le backend est composé de 7 microservices .NET, chacun responsable d'un domaine métier spécifique :

- **User Service** : Gestion des utilisateurs, synchronisation avec Keycloak, communication gRPC.
- **User Content Service** : Gestion du portfolio (projets, compétences, expériences, formations, certifications, langues).
- **CV Service** : Création et versionnement des CV, gestion des templates, export PDF.
- **Workflow Service** : Orchestration de la génération de CV, appel aux agents IA.
- **Application Service** : Suivi des candidatures, communication gRPC avec User Service.
- **Job Offer Service** : Gestion des offres d'emploi, déclenchement du crawling, hub SignalR.
- **Notification Service** : Envoi d'emails (MailKit), rappels planifiés (Hangfire), consommateur Kafka.

La passerelle API (**YARP**) est un service .NET dédié qui agit comme point d'entrée unique. Elle gère l'authentification, le routage vers les microservices et la publication

d'événements Kafka.

La communication événementielle est assurée par **Apache Kafka** en mode KRaft. Le diagramme ci-dessous illustre les principaux événements échangés entre les services via Kafka.



eraser

FIGURE 4.4 – Flux des événements Kafka — Publications et consommations

4.5.2 Pipeline de génération de CV

Le **Workflow Service** orchestre le pipeline de génération de CV, composé de 5 agents IA agissant en séquence. Chaque agent traite une étape spécifique du flux de travail.

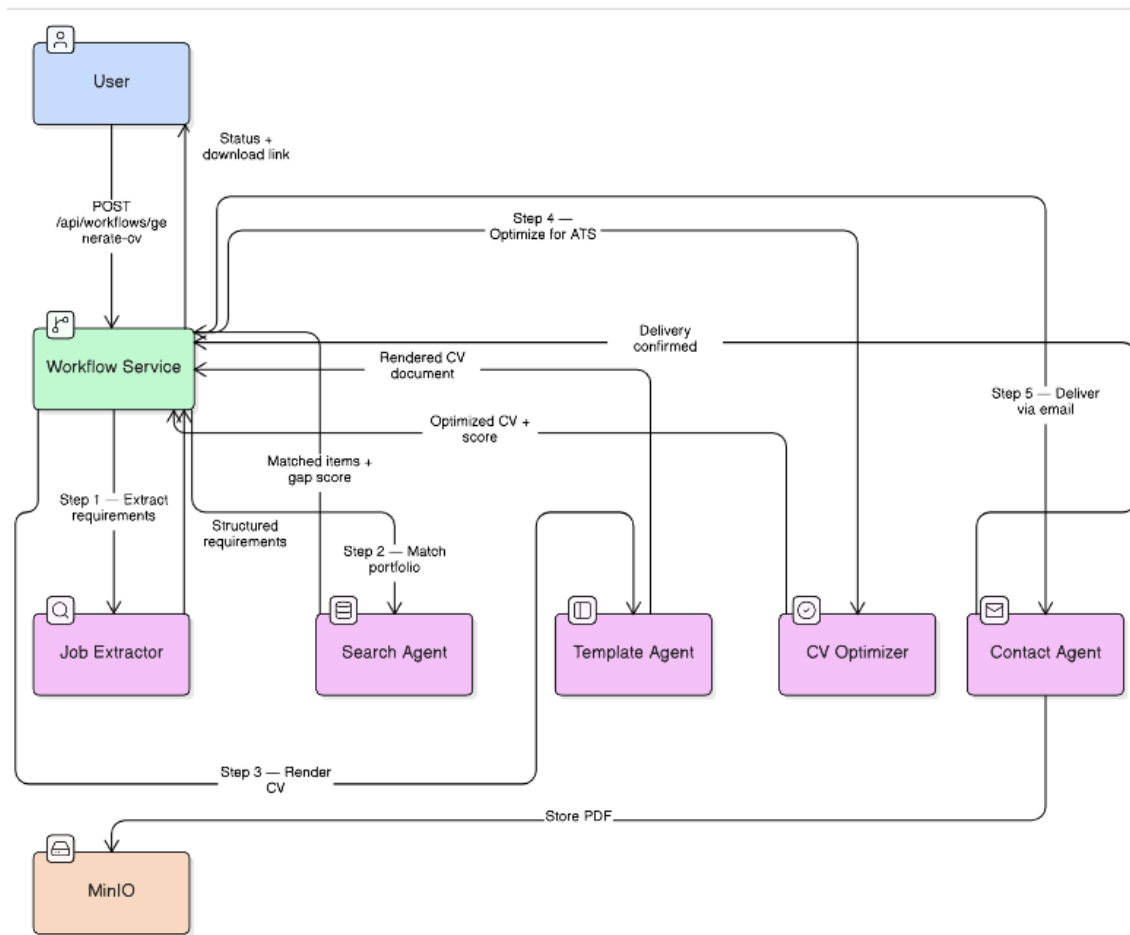


FIGURE 4.5 – Pipeline de génération de CV — De l’offre d’emploi à l’envoi par email

4.5.3 Développement Frontend

Le frontend est développé avec **Angular 21** en utilisant les **standalone components**, **RxJS** pour la gestion asynchrone et les **Signals** pour la réactivité. L’interface suit le design system **Propel** avec des couleurs en espace OKLCH.

Les principales fonctionnalités du frontend incluent :

- Pages de connexion et d’inscription (Keycloak).
- Tableau de bord avec statistiques et graphiques.

- Interface de génération de CV (saisie d’offre d’emploi).
- Vue Kanban pour le suivi des candidatures (drag-and-drop).
- Vue calendrier avec rappels.
- Gestion complète du portfolio.
- Vues liste et analytique des candidatures.

4.5.4 Base de données

L’architecture microservices implique l’utilisation de 8 bases de données **PostgreSQL 16** indépendantes, une par microservice, plus une pour Keycloak. Chaque base utilise l’extension **pgvector** pour le support des recherches vectorielles utilisées par l’agent de matching RAG.

4.6 Déploiement

4.6.1 Mise en place de l’environnement de développement

L’environnement de développement est entièrement conteneurisé. Un fichier **Makefile** à la racine du projet automatise les commandes courantes (démarrage des services, exécution des migrations, lancement des tests). Chaque sous-projet (backend, ai_agents) possède également son propre Makefile.

4.6.2 Dockerisation de l’application

L’ensemble de l’application est dockerisé avec des **Dockerfiles multi-stage** pour optimiser la taille des images de production. Au total, 18 Dockerfiles ont été créés couvrant l’ensemble des services.

Les images sont construites en deux étapes :

1. **Build** : Compilation et génération des artefacts.
2. **Runtime** : Image légère ne contenant que l’exécutable et ses dépendances.

4.6.3 Pipeline CI/CD

Le pipeline CI/CD est implémenté avec **GitHub Actions** et comprend 6 phases séquentielles. Chaque phase doit réussir pour passer à la suivante.

FIGURE 4.6 – Pipeline CI/CD

4.7 Présentation de l'application

4.7.1 Interface utilisateur

FIGURE 4.7 – Interface du tableau de bord

FIGURE 4.8 – Interface de génération de CV

FIGURE 4.9 – Tableau Kanban de suivi des candidatures

FIGURE 4.10 – Interface de gestion du portfolio

4.7.2 Fonctionnalités principales

Le système **Propel** offre les fonctionnalités principales suivantes :

- **Génération intelligente de CV** : Pipeline de 6 agents IA (extraction, matching RAG, template, optimisation ATS, contact) coordonnés par un orchestrateur.
- **Suivi de candidatures** : Vue Kanban avec drag-and-drop, vue liste, vue calendrier, graphiques analytiques.
- **Gestion de portfolio** : CRUD complet pour projets, compétences, expériences, formations, certifications, langues et réseaux sociaux.
- **Crawling d'offres d'emploi** : Agent IA dédié qui collecte automatiquement les offres depuis des plateformes externes.
- **Notifications temps réel** : SignalR pour les mises à jour en direct, Hangfire pour les rappels planifiés, emails via MailKit.

4.7.3 Démonstrations du fonctionnement

4.8 Conclusion

Ce chapitre a présenté en détail la réalisation technique du projet **Propel**. Nous avons couvert l'architecture physique et logicielle, les outils utilisés, le cycle DevSecOps, la gestion du code source, l'implémentation des différentes couches et le déploiement. Le projet démontre une maîtrise complète de la chaîne DevOps, de l'écriture du code jusqu'au déploiement automatisé en passant par la sécurité et la qualité.

Chapitre 5

Conclusion générale et perspectives

5.1 Conclusion générale

Le projet **Propel** nous a permis de concevoir et réaliser une plateforme complète de génération de CV assistée par intelligence artificielle. Ce projet d'intégration a mobilisé l'ensemble des compétences acquises durant notre formation, du développement web à l'architecture microservices, en passant par la conteneurisation, l'intégration continue, la sécurité applicative et l'observabilité.

L'approche DevOps adoptée, avec une organisation en microservices et un pipeline CI/CD complet intégrant qualité et sécurité, a permis de livrer une application robuste et industrialisée. La pipeline d'agents IA basée sur LangChain et LangGraph démontre la capacité à intégrer des technologies avancées d'intelligence artificielle dans une application concrète.

Sur le plan méthodologique, l'adoption de SCRUM avec des réunions quotidiennes et une communication continue via Google Meets et WhatsApp a favorisé une collaboration efficace au sein de l'équipe de six développeurs fullstack, chacun apportant son expertise dans son domaine de responsabilité secondaire.

5.2 Perspectives

Plusieurs perspectives d'amélioration et d'évolution sont envisageables pour **Propel** :

- **Déploiement cloud** : Migrer l'infrastructure vers un fournisseur cloud (AWS, Azure, GCP) avec orchestration Kubernetes pour une meilleure scalabilité.
- **Modèles de langage additionnels** : Intégrer d'autres fournisseurs de LLM (OpenAI, Anthropic, modèles open source en local) pour diversifier les capacités de génération.
- **Internationalisation** : Ajouter le support multilingue pour l'interface et la

génération de CV dans différentes langues.

- **Analyse avancée :** Développer des fonctionnalités d'analyse prédictive pour suggérer les offres d'emploi les plus pertinentes en fonction du profil de l'utilisateur.
- **Mode hors ligne :** Proposer une version allégée de l'application fonctionnant sans connexion Internet pour la consultation des CV et candidatures.
- **Place de marché :** Créer une plateforme de mise en relation entre candidats et recruteurs utilisant les données de profiling générées par l'application.

Bibliographie

Angular documentation, 2025. URL <https://angular.dev>.

Apache kafka documentation, 2025. URL <https://kafka.apache.org/documentation>.

Asp.net core documentation, 2025. URL <https://learn.microsoft.com/en-us/aspnet/core>.

Cypress documentation, 2025. URL <https://docs.cypress.io>.

Docker documentation, 2025. URL <https://docs.docker.com>.

Fastapi documentation, 2025. URL <https://fastapi.tiangolo.com>.

Grafana documentation, 2025. URL <https://grafana.com/docs>.

k6 documentation, 2025. URL <https://grafana.com/docs/k6>.

Keycloak documentation, 2025. URL <https://www.keycloak.org/documentation>.

Langchain documentation, 2025. URL <https://python.langchain.com>.

Postgresql documentation, 2025. URL <https://www.postgresql.org/docs>.

Prometheus documentation, 2025. URL <https://prometheus.io/docs>.

Sonarcloud documentation, 2025. URL <https://docs.sonarcloud.io>.

Terraform documentation, 2025. URL <https://developer.hashicorp.com/terraform/docs>.

Trivy documentation, 2025. URL <https://trivy.dev>.